Nombre: Hernández Silva Roberto Jaziel	**Matrícula**: 1918436
Nombre: Martínez Álvarez Alejandro	**Matrícula**: 2052641
Nombre: Rodríguez Ortiz Danna Saraí	**Matrícula**: 2060308
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Laboratorio guiado. Programa para administrar pedidos de una tienda ficticia TechStore
Fecha: 28/10/2026	

Bibliografia (Formato APA)

- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Facultad de Ingeniería Mecánica y Eléctrica, Universidad Autónoma de Nuevo León. (s. f.). Desarrollo de componentes de negocio: Manual de lectura ilustrado (Unidad 4) [Material de clase].

Objetivo:

Documentar los quince escenarios definidos, señalando sus condiciones iniciales, la operación evaluada, el resultado correspondiente, los Domain Events y las comprobaciones necesarias para demostrar las reglas del programa TechStore.

1. Maquina de estados del pedido

Todos los pedidos se cargan inicialmente como DRAFT. Desde ese estado pueden confirmarse, siempre que tengan artículos y pertenezcan a un cliente activo. Después de confirmarse, el pedido puede enviarse o cancelarse. Si ya fue enviado, puede marcarse como entregado o devuelto.

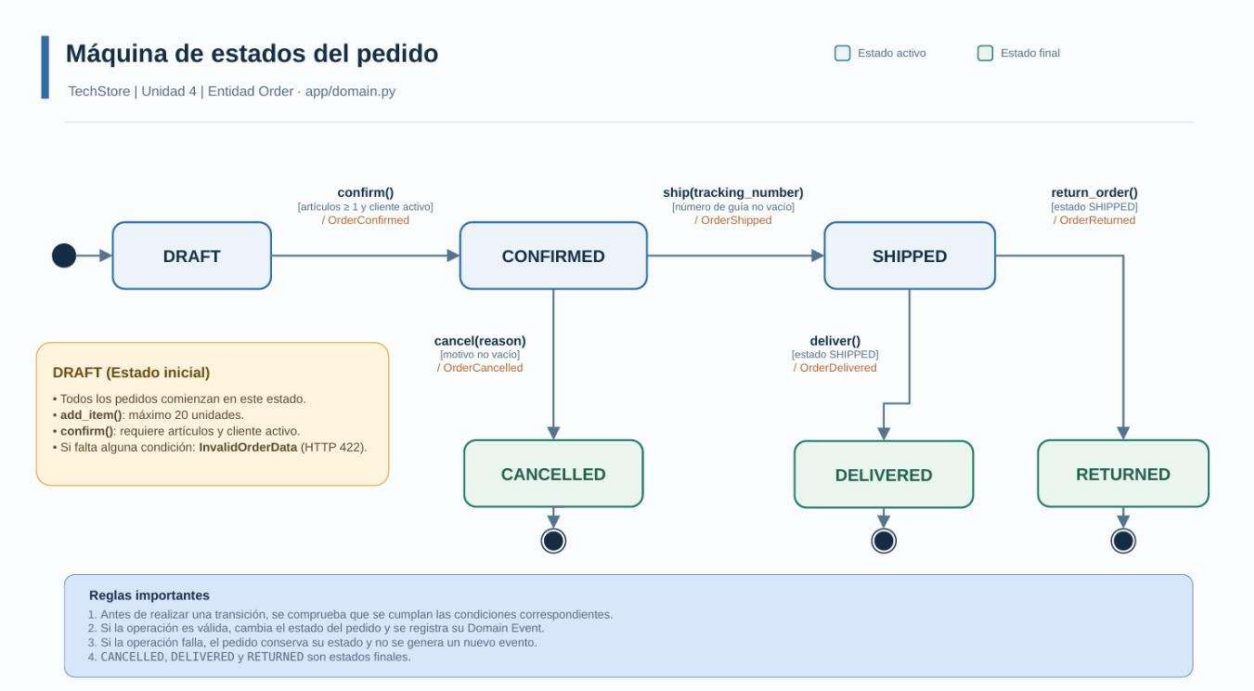


Figura 1. Diagrama de estados elaborado para TechStore, con los métodos y eventos del pedido.

Los tres estados finales son CANCELLED, DELIVERED y RETURNED. No se permite confirmar dos veces un pedido ni cancelarlo cuando ya está en SHIPPED. El diagrama representa las transiciones previstas en el código; las reglas de descuento se evalúan aparte mediante Specification Pattern.

2. Transiciones permitidas

Cada método revisa primero el estado actual y los datos necesarios. Si las condiciones se cumplen, cambia el estado y agrega el evento correspondiente.

Cambio de estado	Método	Condición	Evento
DRAFT → CONFIRMED	confirm()	Debe haber al menos un artículo y cliente activo.	OrderConfirmed
CONFIRMED → SHIPPED	ship(tracking_number)	Guía de envío no vacía.	OrderShipped
CONFIRMED → CANCELLED	cancel(reason)	Motivo de cancelación no vacío.	OrderCancelled
SHIPPED → DELIVERED	deliver()	El pedido tiene que estar en SHIPPED.	OrderDelivered
SHIPPED → RETURNED	return_order()	El pedido tiene que estar en SHIPPED.	OrderReturned

3. Domain Events

Un Domain Event (evento de dominio) deja constancia de algo que ya ocurrió en el pedido. Por ejemplo, OrderShipped se registra cuando el pedido pasó correctamente a SHIPPED, no cuando alguien apenas intentó enviarlo.

En el archivo app/domain.py del equipo, cada evento se crea con los siguientes datos:

Campo	Información que guarda
name	Nombre del evento, por ejemplo, OrderConfirmed.
order_id	Identificador del pedido al que pertenece.
occurred_at	Fecha y hora en que ocurrió, con zona horaria UTC.

4. ¿Qué registra cada evento?

Evento	Cuándo se registra	Qué significa
OrderConfirmed	Al ejecutar confirm() desde DRAFT con datos válidos.	La confirmación fue aceptada y el pedido ya no está en borrador.
OrderShipped	Al ejecutar ship() desde CONFIRMED, con guía.	El pedido cambió a enviado.
OrderCancelled	Al ejecutar cancel() desde CONFIRMED, con motivo.	El pedido se canceló correctamente.
OrderDelivered	Al ejecutar deliver() desde SHIPPED.	El pedido quedó marcado como entregado.
OrderReturned	Al ejecutar return_order() desde SHIPPED.	El pedido quedó marcado como devuelto.

5. ¿En qué momento se genera el evento?

El orden de ejecución es el mismo para las cinco operaciones: primero se revisan las condiciones; después se cambia el estado; y al final se agrega el DomainEvent a la lista events del pedido.

Ejemplo del método confirm() en app/domain.py:

```
if self.status != OrderStatus.DRAFT:
    raise InvalidOrderTransition("Solo DRAFT")
if not self.items or not self.customer.active:
    raise InvalidOrderData("Requiere artículos y cliente activo")
self.status = OrderStatus.CONFIRMED
self.events.append(DomainEvent("OrderConfirmed", self.id))
```

Si alguna validación falla, el método se detiene antes de ejecutar las dos últimas líneas. Por lo tanto, el estado permanece igual y no se agrega un evento nuevo.

Pruebas en test

Prueba en tests/test_domain.py	Qué revisa
test_13_envio_sin_guia	Rechaza ship() sin número de guía después de confirmar.
test_14_confirmacion_doble	Comprueba que la segunda confirmación falla y no añade un segundo evento.
test_15_cancelacion_desde_shipped	Rechaza la cancelación cuando el pedido ya está enviado.

6. Ejemplos de los escenarios

Los siguientes casos están descritos en el reporte de nuestro equipo y corresponden a los datos entregados por el profesor. Se incluyen para mostrar cuándo debe generarse un evento y cuándo no.

Caso	Operación	Estado final	Eventos que corresponden
E03 · O003	Confirmar con cliente inactivo.	DRAFT	Ninguno; la operación se rechaza.
E06 · O007	Confirmar un pedido vacío.	DRAFT	Ninguno; la operación se rechaza.
E11 · O012	Confirmar y luego enviar sin guía.	CONFIRMED	OrderConfirmed; no se agrega OrderShipped.
E12 · O013	Confirmar dos veces.	CONFIRMED	Un solo OrderConfirmed.
E13 · O014	Confirmar, enviar e intentar cancelar.	SHIPPED	OrderConfirmed y OrderShipped; no OrderCancelled.
E14 · O015	Confirmar, enviar y devolver.	RETURNED	OrderConfirmed, OrderShipped y OrderReturned.
E15 · O016	Confirmar, enviar y entregar.	DELIVERED	OrderConfirmed, OrderShipped y OrderDelivered.

7. Relación con API

En el código actual, app/api.py ofrece las operaciones de confirmar, enviar y cancelar. Cuando el estado no permite la acción, la API responde con HTTP 409; si falta un dato obligatorio, como la guía o el motivo, responde con HTTP 422. Si no existe el ID del pedido, responde con HTTP 404.

Los métodos deliver() y return_order() sí aparecen en la clase Order de app/domain.py, pero todavía no tienen un endpoint propio en la API publicada. Para mostrarlos durante la exposición se pueden ejecutar desde Python; no corresponde presentarlos como endpoints implementados.

7. Comprobación de una devolución

Con el proyecto abierto en la carpeta raíz, este ejemplo permite comprobar el caso O015 desde Python sin cambiar los archivos CSV:

```
from cargar_csv import load_fixture
pedido = load_fixture()["O015"]
pedido.confirm()
pedido.ship("GUIA123")
pedido.return_order()
print(pedido.status.value)
print([e.name for e in pedido.events])
```

El resultado esperado es RETURNED y los eventos OrderConfirmed, OrderShipped y OrderReturned, en ese orden.